

Introduction to Data Science and Engineering

- R programming: control flow structures



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo



In this lecture

- Conditional control
- Define functions
- Namespace
- For loop
- `apply` **and** `replicate`

- 
- R is good at:
 - Efficiently perform exploratory data analysis
 - Build data analysis pipelines
 - Prepare data visualization to communicate results
 - R is more than just data analysis:
 - R is primarily a programming language
 - Advanced R programmers can develop complex packages and even improve R itself
 - Learning advanced R is not a target of this course, but an option if you aim to be a “data science software developer” or an “versatile data scientist”
 - Google/GPT syntax that you haven’t learned in the course



Conditional expression

```
library(dslabs)
data(murders)
murder_rate <- murders$total / murders$population*100000

ind <- which.min(murder_rate)

if(murder_rate[ind] < 0.5) {
  print(paste("Hmm...", murders$state[ind]))
} else if (murder_rate[ind] < 1.) {
  print(paste("Sad...", murders$state[ind]))
} else {
  print("Welcome to the U.S.")
}
```

Different from Python:
else if instead of elif

```
> a <- 0
> ifelse(a > 0, 1/a, NA)
[1] NA
```

Python: 1/a if a>0 else NA
C/C++: (a>0)?(1/a):(NULL);
R: ifelse(a > 0, 1/a, NA)

Use `ifelse` to substitute all missing values

```
> data(na_example)
> no_nas <- ifelse(is.na(na_example), 0, na_example)
> sum(is.na(no_nas))
```

`ifelse` operation is vectorized
and very frequently used

```
> na_example
  [1]  2  1  3  2  1  3  1  4  3  2  2 NA  2  2
 [27] NA  2  2  3  1  2  4  1  1  1  4  5  2  3
 [53]  1  1  5  1  3  1 NA  4  4  7  3  2 NA NA
 [79]  3  4  2  3  1  3  2  1  1  1  3  1 NA  3
[105]  3  3  2  2  3  3  3  4  1  1  1  2 NA  4
[131]  1  2  1  3  5  3  2  2 NA NA NA NA  3  5
[157]  6 NA  1  1  2  2  1  3  1  1  5 NA NA  2
[183]  4  1  1  3  1  1 NA NA  3  5  2  2  2  3
[209]  2  1  1 NA  3 NA  1  2  2  1  3  2  2  1
[235]  4  1 NA  5  1  4 NA  3 NA NA  1  1  5  2
[261]  6  2  2  2 NA  2 NA  2 NA  3  3  2  2  4
[287] NA  2  2 NA  1  1  3  2  3  3  1 NA  1  4
```

```
> no_nas
  [1]  2  1  3  2  1  3  1  4  3  2  2  0  2  2  1  4  0  1  1
 [40]  3  4  1  2  4  1  1  2  1  5  0  0  0  1  1  5  1  3  1
 [79]  3  4  2  3  1  3  2  1  1  1  3  1  0  3  1  2  2  1  2
[118]  4  3  4  3  1  2  1  0  0  0  0  1  5  1  2  1  3  5  3
[157]  6  0  1  1  2  2  1  3  1  1  5  0  0  2  4  0  2  5  1
[196]  3  1  2  2  3  2  1  0  2  0  1  0  0  2  1  1  0  3  0
[235]  4  1  0  5  1  4  0  3  0  0  1  1  5  2  3  3  2  4  0
[274]  4  3  1  4  2  0  2  4  0  6  2  3  1  0  2  2  0  1  1
[313]  3  2  1  2  3  5  5  1  2  3  3  1  0  0  1  2  4  0  2
[352]  5  3  2  3  4  1  4  3  1  0  2  1  2  2  1  2  2  6  1
[391]  3  0  3  3  0  2  0  1  2  1  1  4  2  1  4  4  0  1  2
[430]  2  1  1  1  3  2  2  0  4  4  4  1  1  0  4  3  0  1  3
```



all any

```
> z <- c(TRUE, TRUE, FALSE)
> any(z)
[1] TRUE
> all(z)
[1] FALSE
```



Defining functions

```
x <- 1:100

avg <- function(x, arithmetic = TRUE){
  n <- length(x)
  ifelse(arithmetic, sum(x)/n, prod(x)^(1/n))
}

identical(mean(x), avg(x))
```

You can also write
`return(VALUE)` explicitly

prod {base}

R Documentation

Product of Vector Elements

Description

prod returns the product of all the values present in its arguments.

Usage

```
prod(..., na.rm = FALSE)
```

Arguments

... numeric or complex or logical vectors.

na.rm logical. Should missing values be removed?

Test Objects for Exact Equality

Description

The safe and reliable way to test two objects for being *exactly* equal. It returns TRUE in this case, FALSE in every other case.

Usage

```
identical(x, y, num.eq = TRUE, single.NA = TRUE, attrib.as.se
          ignore.bytecode = TRUE, ignore.environment = FALSE,
          ignore.srcref = TRUE, extptr.as.ref = FALSE)
```

Arguments

x, y any R objects.

num.eq logical indicating if (double and complex non-NA) numbers should be compared using == ('equal'), or by bitwise comparison. The latter (non-default) differentiates between -0 and +0.

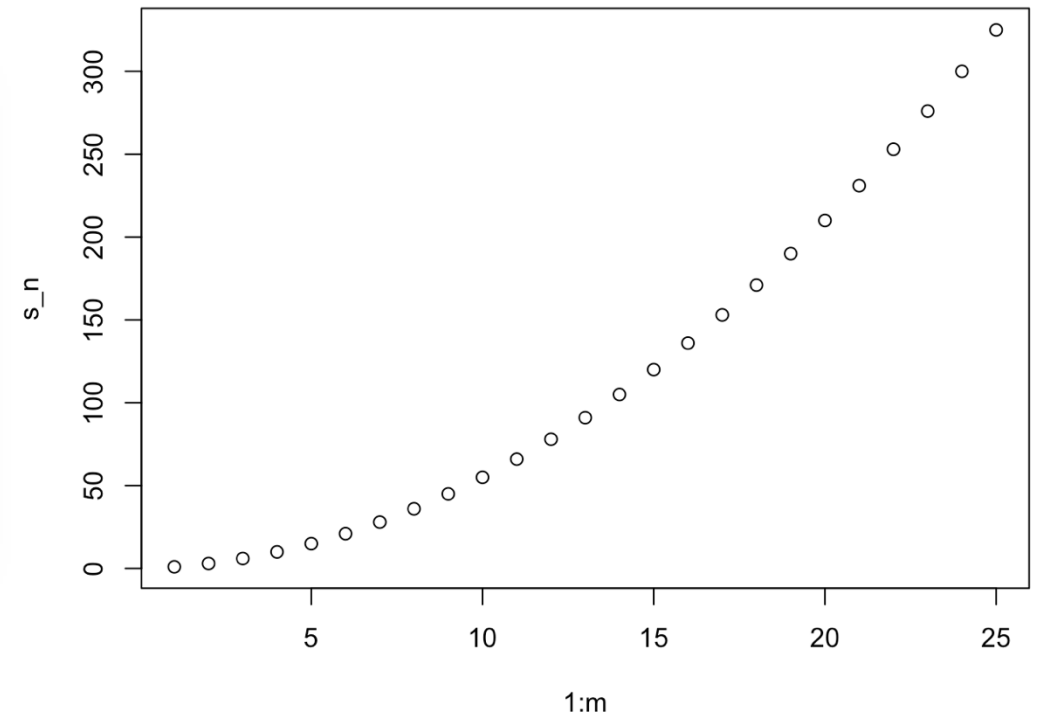
Namespace

```
> library(tidyverse)
— Attaching packages —
✓ ggplot2 3.4.0      ✓ purrr  1.0.0
✓ tibble  3.1.8      ✓ dplyr  1.0.10
✓ tidyr   1.2.1      ✓ stringr 1.5.0
✓ readr   2.1.3      ✓ forcats 0.5.2
— Conflicts —
✗ dplyr::filter() masks stats::filter()
✗ dplyr::lag()    masks stats::lag()
```

`dplyr::filter()` masks `stats::filter()` means:
if you run `filter()`, `dplyr::filter()` is selected;
explicitly type and run `stats::filter()`

For-loop

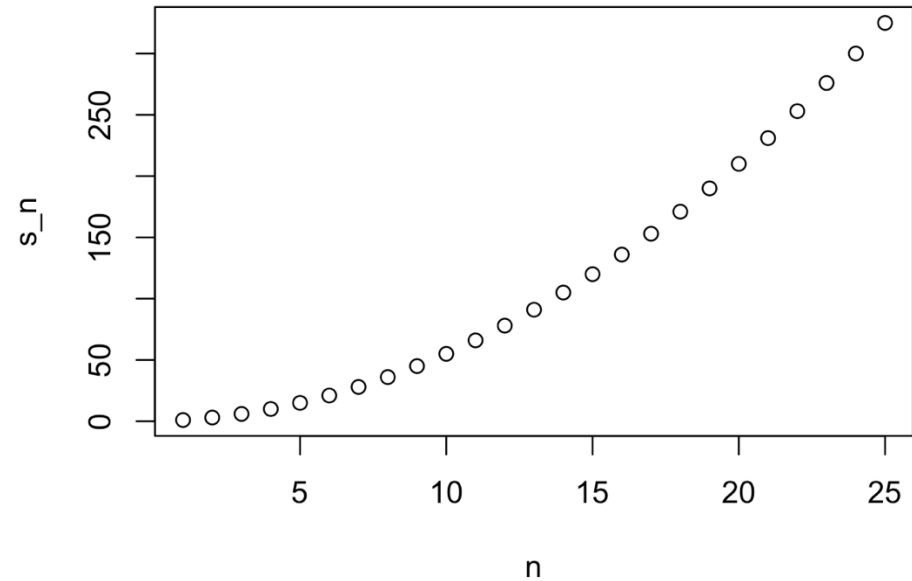
```
compute_s_n <- function(n){  
  x <- 1:n  
  sum(x)  
}  
  
m <- 25  
s_n <- vector(length = m) # create an empty vector  
for(n in 1:m){  
  s_n[n] <- compute_s_n(n)  
}  
  
plot(1:m, s_n)
```



sapply

`sapply()` function takes list, vector or data frame as input and gives output in vector or matrix. It is useful for operations on list objects and returns a list object of same length of original set.

```
compute_s_n <- function(n){  
  x <- 1:n  
  sum(x)  
}  
n <- 1:25  
s_n <- sapply(n, compute_s_n)  
plot(n, s_n)
```





The `apply` family

See `?apply`

Function	Input	Output	Primary Use Case	Dimension Worked On
<code>apply()</code>	Matrix/Array	Vector/Matrix	Apply a function over rows/cols of a matrix	Specified (1=rows, 2=cols)
<code>lapply()</code>	List/Vector	List	Apply a function to each list element	All elements
<code>sapply()</code>	List/Vector	Vector/Matrix	Simplified <code>lapply()</code> output	All elements
<code>vapply()</code>	List/Vector	Pre-specified	<code>sapply()</code> with type safety	All elements
<code>mapply()</code>	Multiple lists	List/Vector	Multivariate apply (parallel application)	All elements



replicate

`replicate` repeats an expression multiple times,
very useful for sampling observations

```
> data <- replicate(n=5, rnorm(10, mean=0, sd=1))  
> data
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1.37095845	1.3048697	-0.3066386	0.45545012	0.2059986
[2,]	-0.56469817	2.2866454	-1.7813084	0.70483734	-0.3610573
[3,]	0.36312841	-1.3888607	-0.1719174	1.03510352	0.7581632
[4,]	0.63286260	-0.2787888	1.2146747	-0.60892638	-0.7267048
[5,]	0.40426832	-0.1333213	1.8951935	0.50495512	-1.3682810
[6,]	-0.10612452	0.6359504	-0.4304691	-1.71700868	0.4328180
[7,]	1.51152200	-0.2842529	-0.2572694	-0.78445901	-0.8113932
[8,]	-0.09465904	-2.6564554	-1.7631631	-0.85090759	1.4441013
[9,]	2.01842371	-2.4404669	0.4600974	-2.41420765	-0.4314462
[10,]	-0.06271410	1.3201133	-0.6399949	0.03612261	0.6556479



In this lecture

- Conditional control
- Define functions
- Namespace
- For loop
- `apply` **and** `replicate`